



Leibniz-Rechenzentrum
der Bayerischen Akademie der Wissenschaften



Introduction to Molecular Modelling and Computational Chemistry on HPC Systems at LRZ

Dr. Ferdinand Jamitzky, Dr. Plamen Dobrev, Dr. Prasanth Ganta, Dr. Birkan Emrem , Dr. Elmira Birang,
Matteo Foglieni (all LRZ)



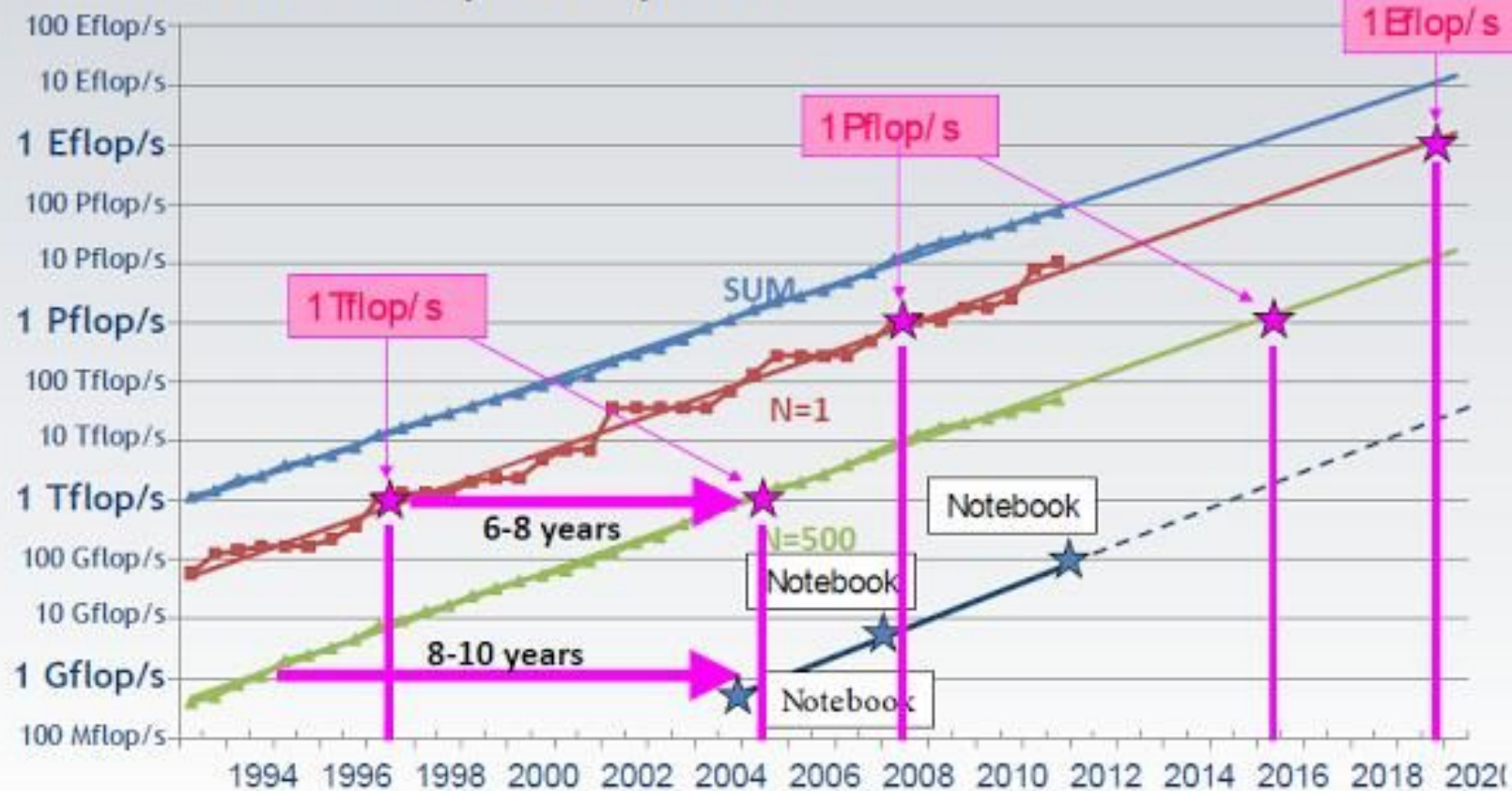
Introduction to Molecular Modelling and Computational Chemistry on HPC Systems at LRZ

Date:	18.11.2025 – 20.11.2025
Location:	online
Contents:	This online course introduces the fundamental principles and key concepts of molecular modeling on LRZ high-performance computing systems. Participants will be introduced to several molecular dynamics software packages and learn how to use them effectively. The training will also provide insights into scientific workflows commonly used in molecular simulation applications. Topics covered include: an overview of molecular simulation software applications, an introduction to visualisation of molecules using VMD, pymol and VESTA, hands-on experience running GROMACS, CP2K, DFBT+ and LAMMPS on LRZ hardware.

Supercomputer scaling

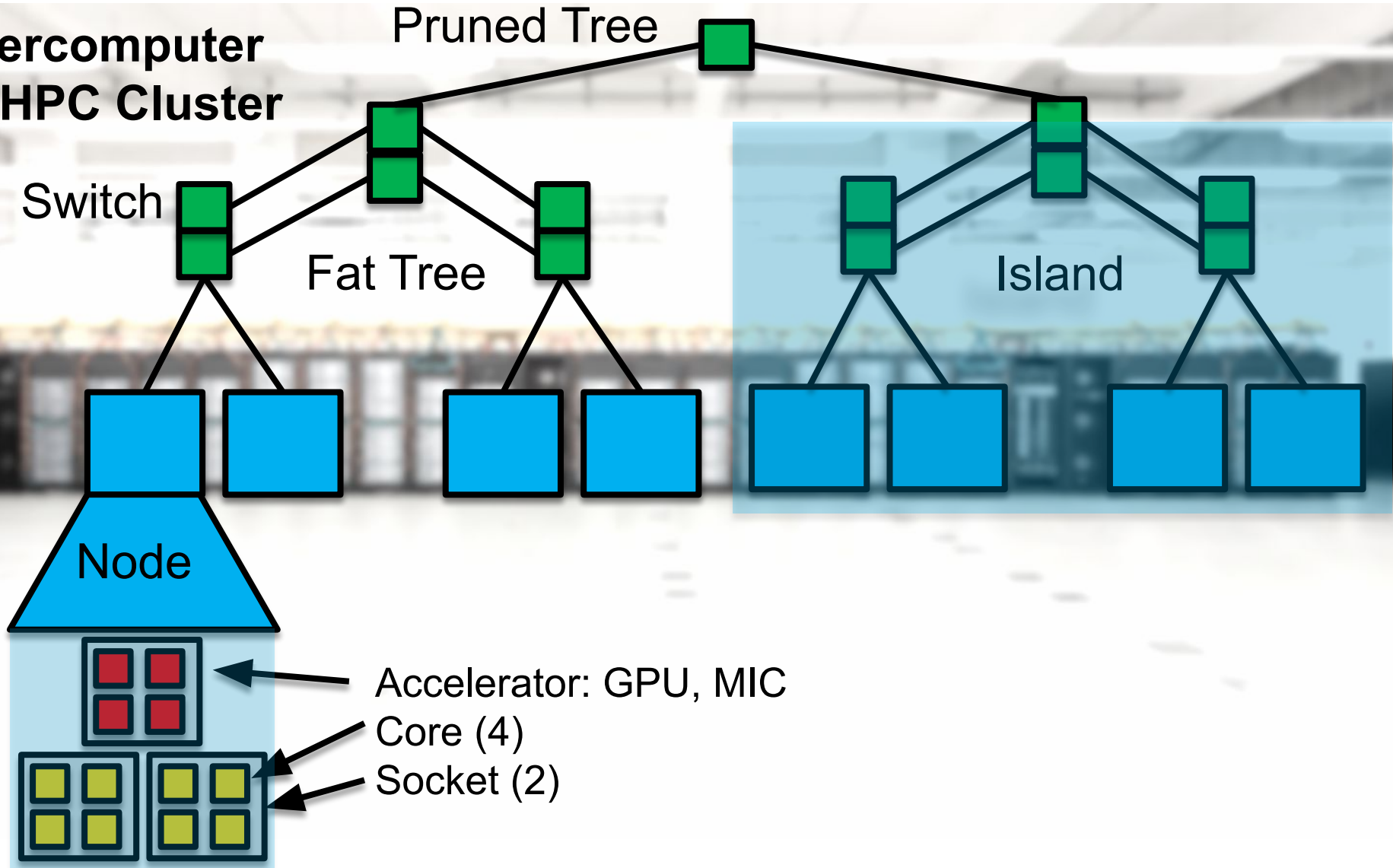
Future of Supercomputing

Moore's Law for Supercomputers



Supercomputer Layout

Supercomputer
aka HPC Cluster





Supercomputing @ LRZ

LRZ HPC, Cloud and AI Systems

CoolMUC

SuperMUC-NG

Compute Cloud

AI Systems

CM4

Phase1

Phase2

GCS Dev
Cloud

RedLion

6000 Nodes
with:
48 Cores
96 GB RAM

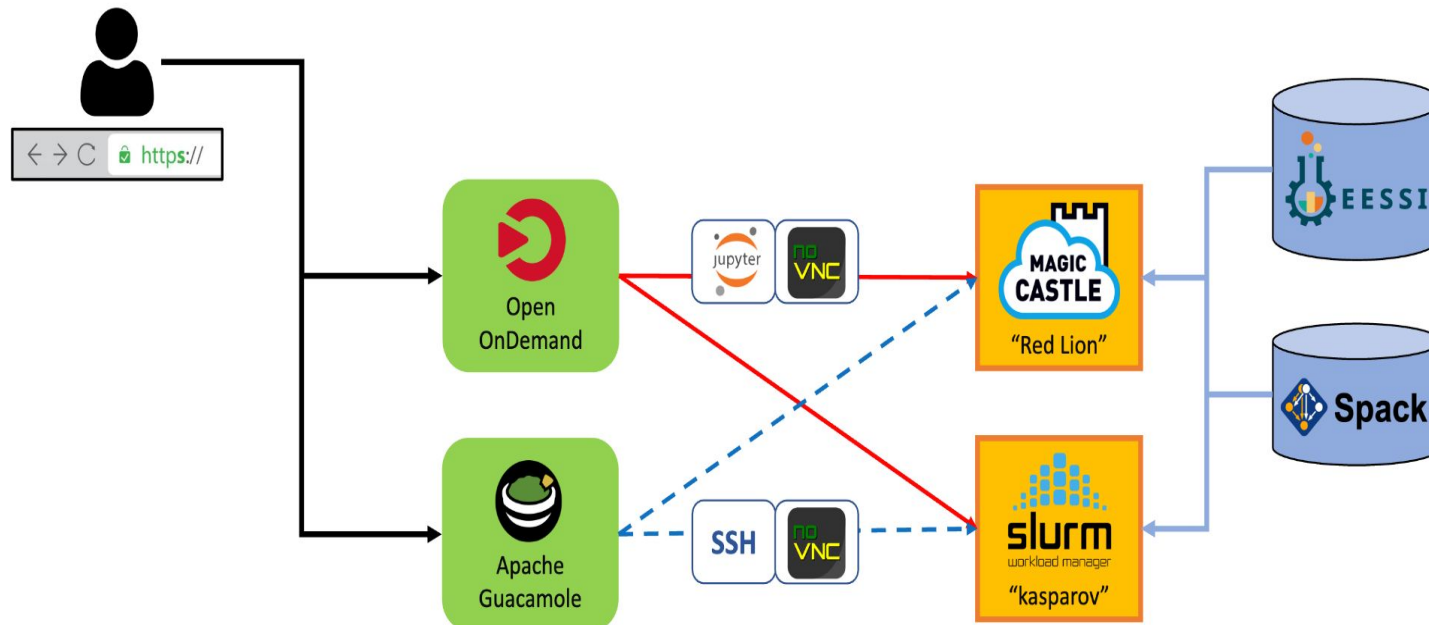
240 Nodes
with:
112 Cores
512 GB RAM
4 GPUs
128 GB RAM

4 Nodes
with:
80 Cores
1 TB RAM

5 GPU
Nodes

Red Lion

Red Lion ist der Kurs- und Test-Server am LRZ. Er ist als virtueller SLURM Cluster auf der Compute Cloud realisiert. EESSI und Spack stellen die Software bereit.



JupyterLab auf OpenOnDemand

The screenshot displays the JupyterLab web interface. The left sidebar shows a file browser with a search bar and a list of files and folders. The central area displays a 'Notebook' section with icons for Python 3 (ipykernel), Julia 1.10.3, Pluto Notebook, R, and a userenv-kernel. The bottom terminal window shows the login prompt and a welcome message from the Leibniz Supercomputing Centre (LRZ) Compute Cloud (CC) for the Gauss Centre for Supercomputing (GCS).

File browser contents:

Name	Modified
cursor_test	last mo.
data	12d ago
f2py.ipynb	last mo.
hello_world.c	2mo ago
hello.txt	2mo ago
helloworld.f90	2mo ago
julia.ipynb	14d ago
matplotlib_test.ipynb	2mo ago
matrix.f90	last mo.
pandas_test.ipynb	2mo ago
pandas2.ipynb	2mo ago
pytorch_test.ipynb	11d ago
README.ipynb	2mo ago
scratch.ipynb	last mo.
test.txt	2mo ago
Untitled.ipynb	last mo.
Untitled1.ipynb	12d ago
Untitled2.ipynb	11d ago

Notebook icons:

- Python 3 (ipykernel)
- Julia 1.10.3
- Pluto Notebook [^]
- R
- userenv-kernel

Terminal output:

```
#####  
#  
# GCS-LRZ-CC  
#  
# Welcome to the Leibniz Supercomputing Centre (LRZ) Compute Cloud (CC)  
# for the Gauss Centre for Supercomputing (GCS).  
#  
# Please read the README.pdf file if this is your first time logging into  
# the system.  
# If you encounter issues opening it (such as Safari for macOS users),  
# right-click to download and read it locally.  
#####  
  
(userenv) jovyan@portalgauss-t43939608fb54  
(userenv) jovyan@portalgauss-t43939608fb54  
(userenv) jovyan@portalgauss-t43939608fb54c86b206629a0dd438e5-6:~$
```



Important SLURM commands

- List my jobs:

```
$ squeue -u <uid>
```

- Cancel my job

```
$ scancel <jobid>
```

- Submit batch job

```
$ sbatch myscript.slurm
```

- Run interactive shell

```
$ salloc -n1 srun --pty bash -i
```



module system

Modules and LRZ Software Stack

<https://doku.lrz.de/environment-modules-10745963.html>

Modules provide an easy mechanism to manage the user environment via `module` command, e.g. making application-specific settings available in the current shell:

<code>module load <package></code>	Load a module	<code>module unload <package></code>	Remove loaded module
<code>module avail</code>	List all available modules	<code>module avail <name></code>	List modules starting with characters "name"
<code>module list</code>	List currently loaded modules	<code>module show <package></code>	Show environment changes when module is loaded
<code>module switch <package></code>	Switch to another package version	<code>module use -p <path></code>	Prepend path to modules search list (-a = append)
<code>module search <keyword></code>	Search for modules containing the keyword	<code>module purge</code>	Clean-up (reset) module environment
<code>module test <package></code>	Test a module	<code>module help</code>	Show usage of module command

Shared Memory (your laptop)

- a few threads working closely together (10-100)
- shared memory
- single tasklist (program)
- cache coherent non-uniform memory architecture aka ccNUMA
- results are kept in shared memory





- many independent processes (10 - 100.000)
- no communication between processes
- individual tasklist for each process
- private memory for each process
- results are stored in a large storage medium



- many independent processes (10 - 100.000)
- one tasklist for all (program)
- everyone can talk to each other (in principle)
- private memory
- needs communication strategy in order to scale out
- very often: nearest neighbor communication
- beware of deadlocks!